Quest 7: The Exploding

Lord of the Wings · impulses spreading (and exploding) through a graph of clouds

> This guide uses the official puzzle text. The story is summarised and the rules are restated in plain words. Every number in the examples and traces was checked by running the code in this repo (`node partN.ts --sample`, plus small instrumented copies to print intermediate values).

In one sentence: fire an impulse into one cloud; it spreads along one-way connections, losing 1 strength per hop and adding its strength to every cloud it lands on. Report the total energy at the end.
Input: one cloud per line, `id=ID, energy=ENERGY, connections=ID_1,ID_2,...`
Output: the total energy of all (remaining) clouds.
What changes between parts: Part I uses strength 5 from the cloud with the most connections. Part II uses strength 10, so the number of impulses explodes and you have to **count** them instead of listing them. Part III adds real explosions: clouds that reach 1,000,000 disappear and fire new strength-10 impulses, and you pick the best starting cloud.

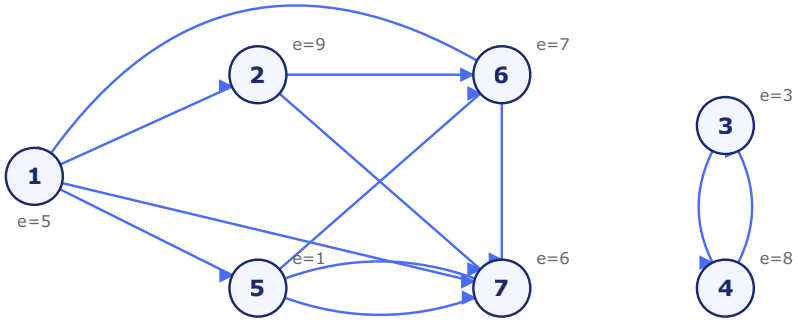
1. The story

The contamination has reached the whole kingdom's food, so the hero goes to an old Academy friend, Sarumonk, for help with dark magic. Sarumonk is already corrupted, still angry about a prank on his graduation robes, cages the team and demands a lightning duel. To win it, the hero has to work out how charge sent into one cloud spreads across the sky, first on practice notes, then for real, when overloaded clouds start bursting.

2. Understanding the question

The clouds form a directed graph

Each cloud is a node, and each `connections` entry is a one-way arrow. Arrows can go back to earlier clouds, form cycles (`3 → 4 → 3` in the sample), or even point at the cloud itself.



The sample clouds (Part I, the first Part II example, Part III). Arrows are one-way connections; *e* is the starting energy.

How an impulse spreads

- The first cloud receives strength *S* (5 in Part I, 10 later) and adds *S* to its energy.
- Any cloud hit by strength *s* > 1 sends a **separate copy** of strength *s* – 1 along **every** outgoing arrow. A strength-1 impulse adds 1 and stops.
- Copies never merge or cancel. Two impulses arriving at the same cloud each add their energy and each send their own copies. So this is **counting paths**: the energy added to a cloud is the sum, over every path of length *k* from the start, of *S* – *k*.
- All impulses move in lockstep: at stage *k* every live impulse has strength *S* – *k*. That's why the puzzle lists the energy "by strength".

Extra examples of my own (all run through the real code)

Clouds (all start at energy 0)	Part I (strength 5)	Part II (strength 10)	Why it's interesting
`1 → 1` (a self-loop)	15	55	The impulse keeps landing on cloud 1: 5+4+3+2+1 = 15 and 10+9+…+1 = 55.
`1 → 2`, `2 → 1`	15	55	Same total, split as 5+3+1 = 9 on cloud 1 and 4+2 = 6 on cloud 2.
`1 → 2,3`, `2 → 4`, `3 → 4`, cloud 4 has `connections=` (empty)	19	44	A diamond: cloud 4 is reached along two paths and gets 3 + 3 = 6 in Part I. The empty list is a parsing edge case (see below).

Shared code: reading the clouds (identical in all three parts)

```
import { readFileSync } from "node:fs";

const isSample = process.argv.includes("--sample");
const file = isSample ? "sample1.txt" : "input1.txt";
const INPUTS: string[] = readFileSync(file, "utf8").trim().split("\n");

const clouds: {
  id: number;
  energy: number;
  connections: number[];
}[] = [];

INPUTS.forEach((i) => {
  const [id = "", energy = "", connections = ""] = i
    .split(" ")
    .map((s) => s.split("=").slice(1).join("=").replace(/,$/, ""));

  clouds.push({
    id: Number(id),
    energy: Number(energy),
    connections: connections.split(",").map((c) => Number(c)),
  });
});
```

q7/part1.ts, lines 1–23

Lines	What happens
3–5	`--sample` picks the sample file; otherwise the real input. The file is split into lines.
7–11	Each cloud becomes `{{ id, energy, connections }}` in the `clouds` array.
14–16	Split the line at spaces: `"id=1,"`, `"energy=5,"`, `"connections=2,5,7"`. For each piece, keep what's after the first `=` (`slice(1).join("=")`) and strip a trailing comma. Destructuring with defaults gives `id = "1"`, `energy = "5"`, `connections = "2,5,7"`.
18–22	Convert to numbers, and split the connection list at commas.

> **Empty connection list (lucky behaviour):** if a cloud had `connections=` with nothing after it, `"".split(",")` gives `[""]`, and `Number("")` is `0`. So the cloud would get a fake connection to cloud `0`. It still works, because there's no cloud 0 and the lookup skips it (Part I line 40, Part II line 46, Part III line 59). That's how the diamond example above gives the right 19. Your real input has no empty lists, so this never comes up.

3. Part I: one impulse of strength 5

The question

Send strength 5 into the cloud with the most outgoing connections, let it spread completely, and add up the energy of all clouds (including the ones never reached).

Worked example (sample)

Cloud 1 has three connections, so it's the start. Grouping the hits by strength:

Strength	Clouds hit (number of impulses)	Energy added
5	1 (×1)	5
4	2 (×1), 5 (×1), 7 (×1)	12
3	5 (×1), 6 (×2), 7 (×2)	15
2	1 (×2), 5 (×2), 6 (×1), 7 (×3)	16
1	1 (×1), 2 (×2), 5 (×5), 6 (×2), 7 (×5)	15

The clouds start with 5+9+3+8+1+7+6 = 39 energy, and 5+12+15+16+15 = 63 is added: 39 + 63 = **102**, matching the puzzle.

Step-by-step solution

1. Read every cloud's id, energy and connection list.
2. Pick the cloud with the most connections and add 5 to it.
3. Make a list of impulses: one entry per connection of the start cloud.
4. For strengths 4, 3, 2, 1: add the strength to the cloud of every impulse in the list, and build the next list from all their connections.
5. Print the sum of all clouds' energy.

My solution

```
25  const mostConnectionsCloud = clouds.sort(
26    (a, b) => b.connections.length - a.connections.length,
27  )[0];
28
29  const INITIAL_ENERGY = 5;
30
31  let connections: number[] = mostConnectionsCloud!.connections;
32  mostConnectionsCloud!.energy += INITIAL_ENERGY;
33
34  for (let i = INITIAL_ENERGY - 1; i > 0; i--) {
35    const newConnections: number[] = [];
36
37    connections.forEach((c) => {
38      const idx = clouds.findIndex((cloud) => cloud.id === c);
39
40      if (idx < 0) return;
41
42      clouds[idx]!.energy += i;
43      newConnections.push(...clouds[idx]!.connections);
44    });
45
46    connections = newConnections;
47  }
48
49  const totalEnergy = clouds.reduce((prev, current) => prev + current.energy, 0);
50
51  console.log(totalEnergy);
```

q7/part1.ts, lines 25–51

Lines	What happens
25–27	Sort the clouds by number of connections, most first, and take the first one. `sort` reorders `clouds` in place, which is harmless here because only the total is needed.
31–32	Add 5 to the start cloud. `connections` now holds the list of clouds the strength-4 impulses go to: one entry per impulse.
34	One loop round per stage, with `i` = 4, 3, 2, 1 being the strength of the impulses in that stage.
37–44	For each impulse in the list, find its cloud, add `i`, and push that cloud's connections onto the next list. If a cloud is reached twice, its connections are pushed twice: one copy per impulse, exactly as the rules say.
46	The next stage works on the impulses produced by this one.
49–51	Sum the energy of every cloud and print it.

Trace: the impulse list at each stage (sample, from an instrumented copy)

`i`	`connections` at the start of the round	Energies after the round (clouds 1–7)
(start)	—	10, 9, 3, 8, 1, 7, 6
4	`[2,5,7]`	10, 13, 3, 8, 5, 7, 10
3	`[6,7,6,7,5]`	10, 13, 3, 8, 8, 13, 16
2	`[1,7,5,1,7,5,6,7]`	14, 13, 3, 8, 12, 15, 22
1	`[2,5,7,5,6,7,2,5,7,5,6,7,1,7,5]`	15, 15, 3, 8, 17, 17, 27

The final energies 15, 15, 3, 8, 17, 17, 27 are exactly the puzzle's, summing to 102.

> **The list grows fast:** the list holds every impulse separately, so its length is the number of paths, which multiplies by the average number of connections at every stage (3 → 5 → 8 → 15 here). In the last round (`i = 1`), lines 37–44 also build a 26-entry list that is never used, since strength-1 impulses don't travel on. At strength 5 this is fine; at strength 10 it wouldn't be, which is what Part II is about.

4. Part II: strength 10

The question

Same process, but the first impulse has strength 10, so there are ten stages (10, 9, …, 1).

Worked examples

On the Part I clouds, the start is still cloud 1, and after ten stages the energies are 152, 92, 3, 8, 344, 249, 468, total **1316**. On the denser second example (every cloud has 3–6 connections, cloud 2 has the most), the total is **1214633**. In that example, 134,638 separate impulses reach cloud 2 in the strength-1 stage alone, and listing them one by one like Part I would mean millions of array entries.

The key idea: group identical impulses

Two impulses with the same strength at the same cloud behave identically from then on. So instead of a list of impulses, keep a **count per cloud**: "this stage, 5 impulses are at cloud 7". A cloud hit by n impulses gains $n \times$ strength and sends n impulses along each connection. The work per stage is now proportional to the number of clouds and connections, not the number of paths.

Step-by-step solution

1. Read the clouds and pick the start as in Part I; add 10 to it.
2. Instead of a list, keep a count of impulses per cloud: start with 1 per connection of the start cloud.
3. For strengths 9 down to 1: each cloud with n impulses gains $n \times$ strength, and adds n to the next count of every cloud it connects to.
4. Print the sum of all clouds' energy.

My solution

Lines 1–27 are the shared code and the start-cloud choice from Part I (with `sample2.txt` / `input2.txt` on line 4).

```
29  const INITIAL_ENERGY = 10;
30
31  let connectionCounts: Map<number, number> = new Map();
32
33  mostConnectionsCloud!.connections.forEach((c) => {
34    const currentCount = connectionCounts.get(c) || 0;
35    connectionCounts.set(c, currentCount + 1);
36  });
37
38  mostConnectionsCloud!.energy += INITIAL_ENERGY;
39
40  for (let i = INITIAL_ENERGY - 1; i > 0; i--) {
41    const newConnectionCounts: Map<number, number> = new Map();
42
43    for (let [id, count] of connectionCounts) {
44      const idx = clouds.findIndex((cloud) => cloud.id === id);
45
46      if (idx < 0) continue;
47
48      clouds[idx]!.energy += i * count;
49      clouds[idx]!.connections.forEach((c) => {
50        const currentCount = newConnectionCounts.get(c) || 0;
51        newConnectionCounts.set(c, currentCount + count);
52      });
53    }
54
55    connectionCounts = newConnectionCounts;
56  }
57
58  const totalEnergy = clouds.reduce((prev, current) => prev + current.energy, 0);
59
60  console.log(totalEnergy);
```

q7/part2.ts, lines 29–60 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
29	Strength 10 now.
31–36	`connectionCounts` maps *cloud id → number of impulses arriving there*. It starts with one impulse per connection of the start cloud. (If the start listed the same connection twice, it would correctly get a count of 2.)
38	The start cloud gets 10.
40	Stages with strength `i` = 9, 8, …, 1.
43–48	For each cloud with `count` impulses, add `i × count`: the same as Part I adding `i` `count` times.
49–52	Each of those `count` impulses sends one copy along every connection, so each target's count in the next map goes up by `count`.
55	Move to the next stage.

Trace: `connectionCounts` on the Part I clouds (from an instrumented copy)

`i`	`connectionCounts` (cloud: count)	Energies after (clouds 1–7)
9	2:1, 5:1, 7:1	15, 18, 3, 8, 10, 7, 15
8	6:2, 7:2, 5:1	15, 18, 3, 8, 18, 23, 31
7	1:2, 7:3, 5:2, 6:1	29, 18, 3, 8, 32, 30, 52
6	2:2, 5:5, 7:5, 6:2, 1:1	35, 30, 3, 8, 62, 42, 82
5	6:7, 7:10, 5:6, 1:2, 2:1	45, 35, 3, 8, 92, 77, 132
4	1:7, 7:16, 5:12, 6:7, 2:2	73, 43, 3, 8, 140, 105, 196
3	2:7, 5:23, 7:28, 6:14, 1:7	94, 64, 3, 8, 209, 147, 280
2	6:30, 7:51, 5:35, 1:14, 2:7	122, 78, 3, 8, 279, 207, 382
1	1:30, 7:86, 5:65, 6:42, 2:14	152, 92, 3, 8, 344, 249, 468

The rows for `i` = 9, 8, 7, 6 are Part I's stages 4, 3, 2, 1 shifted by 5: the counts 2:1, 5:1, 7:1 then 6:2, 7:2, 5:1, and so on are the same groupings as in Part I's table. The last row matches the puzzle's 152, 92, 3, 8, 344, 249, 468.

> **Cost:** each stage loops over at most all clouds, and for each one does a `findIndex` over the array (line 44) plus a loop over its connections. A `Map` from id to cloud would make the lookup instant, but with ten stages it doesn't matter.

5. Part III: explosions

The question

Now a cloud that reaches 1,000,000 energy **explodes**, in this order:

1. All impulses of the current stage land and pass on their weaker copies as usual (even from clouds about to explode).
2. Every cloud now at or above the limit explodes: it disappears, its energy no longer counts, and it receives nothing ever again.
3. Each exploded cloud sends a fresh **strength-10** impulse along each connection. These travel in the next stage together with the ordinary, weaker impulses.

Try every cloud as the start and report the highest total energy of the clouds that survive. (In the sample, the limit is 100.)

Worked example (sample, limit 100)

Start cloud	1	2	3	4	5	6	7
Total left	108	**187**	94	94	101	11	159

Starting at cloud 2: cloud 7 explodes at 135, then clouds 5 and 6 explode in the next stage, leaving 94 + 82 + 3 + 8 = **187**. Cloud 1 has the most connections, but starting there gives only 108.

Extra examples of my own (limit 100, run through a copy of the real code)

Clouds	Start 1	Start 2	What happens
`1 → 2` (e=0), `2 → 1` (e=80)	38	31	Starting at 1, cloud 2 reaches 101 in the strength-5 stage and explodes. Its new strength-10 impulse and the ordinary strength-4 one both land on cloud 1 together in the next stage: 24 + 4 + 10 = 38.
`1 → 2`, `2 → 1` (both e=0)	55	55	10 + 9 + … + 1 = 55 is shared between the two clouds, and neither gets near the limit, so nothing explodes.

Why Part II's map isn't enough any more

In Parts I and II, every impulse in a stage has the same strength. After an explosion, strength-10 impulses travel next to, say, strength-3 ones, so one stage can hold **several strengths at once**. Your code fixes this by keying the map on *cloud and strength*: `"5:10"` means "impulses of strength 10 arriving at cloud 5".

Step-by-step solution

1. Read the clouds. For every cloud as the start, run steps 2–5 on a fresh copy.
2. Add 10 to the start; count strength-9 impulses along its connections, keyed by cloud *and* strength.
3. Each stage: every live cloud with impulses gains strength × count, and (if strength > 1) passes count copies of strength – 1 along its connections.
4. Then every cloud hit this stage that is at or above the limit sends strength-10 impulses along its connections and is removed.
5. Repeat until no impulses are left, sum the surviving clouds, and keep the best total over all starts.

My solution

Lines 1–25 are the shared code (with `sample3.txt` / `input3.txt` on line 4).

```
7  const MAX_ENERGY = isSample ? 100 : 1000000;
```

q7/part3.ts, lines 7–7 (highlighted lines are the ones that changed from the previous part)

```
const INITIAL_ENERGY = 10;

const addImpulse = (
  map: Map<string, number>,
  id: number,
  strength: number,
  count: number,
) => {
  const key = `${id}:${strength}`;
  map.set(key, (map.get(key) || 0) + count);
};

const getTotalEnergy = (initialCloudId: number, allClouds: typeof clouds) => {
  let connectionCounts: Map<string, number> = new Map();

  const initialCloud = allClouds.find((c) => c.id === initialCloudId);
  if (!initialCloud) return 0;

  initialCloud.connections.forEach((c) => {
    addImpulse(connectionCounts, c, INITIAL_ENERGY - 1, 1);
  });

  initialCloud.energy += INITIAL_ENERGY;

  while (connectionCounts.size > 0) {
    const newConnectionCounts: Map<string, number> = new Map();
    const hitClouds: Set<number> = new Set();

    for (const [key, count] of connectionCounts) {
      const [id = 0, strength = 0] = key.split(":").map(Number);
      const cloud = allClouds.find((cloud) => cloud.id === id);

      if (!cloud) continue;

      cloud.energy += strength * count;
      hitClouds.add(id);

      if (strength - 1 > 0) {
        cloud.connections.forEach((c) => {
          addImpulse(newConnectionCounts, c, strength - 1, count);
        });
      }
    }

    for (const id of hitClouds) {
      const idx = allClouds.findIndex((cloud) => cloud.id === id);

      if (allClouds[idx]!.energy < MAX_ENERGY) continue;

      allClouds[idx]!.connections.forEach((c) => {
        addImpulse(newConnectionCounts, c, INITIAL_ENERGY, 1);
      });

      allClouds.splice(idx, 1);
    }

    connectionCounts = newConnectionCounts;
  }
```

```
85
86    const totalEnergy = allClouds.reduce(
87      (prev, current) => prev + current.energy,
88      0,
89    );
90
91    return totalEnergy;
92  };
93
94  let highestTotalEnergy = 0;
95
96  clouds.forEach((c) => {
97    const totalEnergy = getTotalEnergy(c.id, JSON.parse(JSON.stringify(clouds)));
98
99    if (totalEnergy > highestTotalEnergy) highestTotalEnergy = totalEnergy;
100 });
101
102 console.log(highestTotalEnergy);
```

q7/part3.ts, lines 27–102 (highlighted lines are the ones that changed from the previous part)

Lines	What happens
7	The explosion limit: 100 for the sample, 1,000,000 for the real input.
29–37	`addImpulse` adds `count` impulses of a given strength at a given cloud, using the string key `"id:strength"`.
39–49	`getTotalEnergy` runs one whole duel from `initialCloudId`. It seeds strength-9 impulses along the start's connections and adds 10 to the start cloud, like Part II.
51	Instead of a fixed ten stages, keep going while any impulse is alive. Explosions can start new strength-10 impulses late, so the number of stages isn't known in advance.
53	`hitClouds` records which clouds were hit this stage. Only they can have crossed the limit.
55–62	Unpack each key into cloud and strength. If the cloud is gone (exploded, line 80), the impulses are dropped (line 59). Otherwise add `strength × count` and mark the cloud as hit.
64–68	If the strength is above 1, pass `count` copies of strength − 1 along each connection. This happens for every hit cloud, **before** anything explodes, which is rule 1.
71–81	Rule 2 and 3: after the whole stage, every hit cloud at or above the limit sends one strength-10 impulse along each connection (into the same next-stage map, so they travel together with the ordinary copies) and is removed with `splice`.
83	Next stage.
86–91	Sum the energy of the clouds that are left.
94–100	Try every cloud as the start and keep the best total. `JSON.parse(JSON.stringify(clouds))` is a **deep copy**: `getTotalEnergy` changes energies and removes clouds, so every run needs its own fresh copy of the sky.

Trace: start at cloud 2 (sample, limit 100, from an instrumented copy)

Stage	Impulses arriving (`cloud:strength` ×count)	Explodes	Energies after (surviving clouds)
start	cloud 2 gets 10		1:5, 2:19, 3:3, 4:8, 5:1, 6:7, 7:6
1	`6:9` ×1, `7:9` ×1		1:5, 2:19, 5:1, 6:16, 7:15
2	`1:8` ×1, `7:8` ×1, `5:8` ×1		1:13, 2:19, 5:9, 6:16, 7:23
3	`2:7` ×1, `5:7` ×2, `7:7` ×2, `6:7` ×1		1:13, 2:26, 5:23, 6:23, 7:37
4	`6:6` ×3, `7:6` ×4, `5:6` ×2, `1:6` ×1		1:19, 2:26, 5:35, 6:41, 7:61
5	`1:5` ×3, `7:5` ×6, `5:5` ×5, `6:5` ×2, `2:5` ×1		1:34, 2:31, 5:60, 6:51, 7:91
6	`2:4` ×3, `5:4` ×9, `7:4` ×11, `6:4` ×6, `1:4` ×2	7 (at 135)	1:42, 2:43, 5:96, 6:75
7	`6:3` ×12, `7:3` ×20, `5:3` ×13, `1:3` ×6, `2:3` ×2, `5:10` ×1	6 (111), 5 (145)	1:60, 2:49
8	`1:2` ×12, `7:2` ×33, `6:2` ×15, `2:2` ×6, `5:2` ×6, `6:9` ×1, `7:9` ×1, `1:10` ×1, `7:10` ×2, `6:10` ×1		1:94, 2:61
9	`2:1` ×12, `5:1` ×12, `7:1` ×18, `6:1` ×6, `2:9` ×1, `5:9` ×1, `7:9` ×1		1:94, 2:82
10	`6:8` ×1, `7:8` ×1		1:94, 2:82

Clouds 3 and 4 (energy 3 and 8) are never reached and are left out of the last column. Things to notice:

- Stage 6: cloud 7 receives 11 impulses of strength 4 (reaching 135) and still passes them on as 11 strength-3 impulses to cloud 5 *before* exploding. With 2 more from cloud 1, that's the `5:3` ×13 in stage 7. The explosion itself adds the single `5:10`.
- Stage 7 shows two strengths at once (`5:3` and `5:10`), which is why the key includes the strength.
- From stage 8 on, impulses aimed at the exploded clouds 5, 6 and 7 are dropped by line 59. The loop ends after stage 10, when the last strength-8 impulses land on dead clouds and produce nothing. Final total: 94 + 82 + 3 + 8 = 187.

> **The starting cloud's own +10 is never checked for an explosion.** Line 49 adds 10 to the start cloud, but only clouds in `hitClouds` are checked (line 71), and the start is only added there if an impulse comes back to it later. By the rules, the very first impulse is a stage too, so a start cloud pushed over the limit should explode right away. Example with limit 100: `1 → 2` (e=95), `2 → 3`, `3 → 2` (both e=1). Starting at cloud 1, your code reports 152, keeping cloud 1 at 105. Working it out by the rules, cloud 1 should explode at once, giving 102.
> This can only matter if a cloud starts within 10 of the limit. Your real input has no such cloud (checked), so your answer is not affected.

> **Why the loop always ends:** each cloud can explode at most once, and every impulse loses 1 strength per stage. So after the last explosion, at most ten more stages can happen before every impulse has died out.

> **Cost:** one full simulation per starting cloud, each a few dozen stages over a map of at most (clouds × 10) keys. On the real input the whole thing runs in about 0.2 seconds.

6. Summary

Part	Start	Strength	Explosions?	How your code tracks impulses	Sample
I	most connections	5	no	a list with one entry per impulse	102
II	most connections	10	no	`Map` cloud → count of impulses	1316 / 1214633
III	every cloud, keep best	10	yes (limit 1,000,000; 100 in the sample)	`Map` "cloud:strength" → count, run until empty	187

Key ideas: the energy a cloud gets is a sum over **paths**, and paths multiply quickly; impulses that are in the same state (same cloud, same strength) behave identically, so **count them instead of listing them** (the same trick as "lanternfish"-style puzzles); and when the rules say "first everything moves, then everything explodes", process the stage in two passes.